Polarion RT SDK

法律声明

未发布作品。© 2023 西门子

本材料包含西门子工业软件有限公司（Siemens Industry Software, Inc.）及其子公司或关联公司（统称"西门子"）或其许可方所拥有的商业秘密或其他机密信息。对本信息的访问和使用严格受限于客户与西门子之间适用协议的规定。未经西门子明确书面许可，本材料不得在客户设施之外复制、分发或以其他方式披露，也不得以西门子未明确授权的任何方式使用。本文档仅供参考和指导目的。西门子保留随时更改本出版物中包含的规格和其他信息的权利，读者应在所有情况下向西门子咨询以确认是否有任何变更。本文档中关于产品、功能或技术特性的陈述构成技术信息，而非保证或担保，不应对西门子产生任何责任。西门子不承担所有保证责任，包括但不限于适销性和特定用途适用性的默示保证。特别地，西门子不保证产品的运行不会中断或没有错误。

西门子产品销售和许可的条款和条件在西门子与其客户之间的书面协议中规定。西门子的最终用户许可协议和通用合同协议可在以下网址查看：https://www.sw.siemens.com/en-US/sw-terms/

商标：本文使用的商标、标识和服务标记（统称"标记"）为西门子或其他方的财产。未经西门子或标记所有者的书面同意（视情况而定），任何人不得使用这些标记。本文中第三方标记的使用并非旨在表明西门子是产品来源，而是旨在表明产品来自或与特定第三方相关。西门子商标列表可在以下网址查看：https://www.plm.automation.siemens.com/global/en/legal/trademarks.html。

注册商标 Linux® 根据 LMI 的子许可使用，LMI 是该标记世界范围内所有者 Linus Torvalds 的独家被许可人。

关于西门子数字化工业软件

西门子数字化工业软件是全球领先的产品生命周期管理（PLM）软件和服务提供商，拥有 700 万个许可席位和全球 71,000 家客户。西门子数字化工业软件总部位于德克萨斯州普莱诺，与企业协作提供开放解决方案，帮助他们将更多创意转化为成功的产品。有关西门子数字化工业软件产品和服务的更多信息，请访问：https://www.siemens.com/plm

支持中心：https://www.support.sw.siemens.com

文档反馈提交：https://www.support.sw.siemens.com/doc_feedback_form

概述

- 概述
- 自定义 Collector（采集器）示例
- 工作区准备
- 创建项目插件
- 实现
- 部署到已安装的 Polarion
- 从工作区执行
- 配置
- 自定义 Parser（解析器）示例
- 工作区准备
- 创建项目插件
- 实现
- 部署到已安装的 Polarion
- 从工作区执行
- 配置

概述

Resource Traceability（资源可追溯性）赋予 Polarion 用户对源代码资产进行语义链接的能力（将 Work Item（工作项）直接链接到实现它们的源代码函数、方法或变量）。它还可用于解析与编程语言无关的不同格式的文件。资源文件的处理分为两步：第一步，从仓库（repository）中收集可解析的资源文件；第二步，由兼容的解析器对收集的文件进行解析。Resource Traceability 功能的这两个组件分别称为 Collector（采集器）和 Parser（解析器）。Collector 负责连接到特定类型的仓库（如 SVN 或 Git），并收集自上次采集运行以来更新的文件。Parser 负责解析特定编程语言或文件格式的文件语法，识别语义元素和相关链接，并将它们返回给采集引擎。采集引擎将已解析文件中的链接持久化到内部存储中。

Resource Traceability SDK 提供了一种开发自定义 Parser 和 Collector 组件的方法。开发者应实现 `com.siemens.polarion.rt.collectors.api.RtCollector` 接口来定义自定义 Collector，实现 `com.siemens.polarion.rt.parsers.api.RtParser` 接口来定义自定义 Parser。开发的组件通过插件清单文件 `plugin.xml` 中的 extension（扩展）元素暴露给 Polarion。请查看 javadoc 获取更多详细信息。

自定义 Collector 示例

工作区准备

请参阅 Polarion SDK 主指南中的"工作区准备"章节。

创建项目插件

1. 选择 **File > New > Java Project**。
2. 在出现的对话框中，输入新的插件项目名称，然后点击 **Next** 按钮。
3. 在 **Projects** 选项卡中，将所需的项目添加到构建路径中。插件 `com.siemens.polarion.rt.api`、`com.polarion.platform.repository` 和 `com.polarion.subterra.base` 是开发 Collector 项目所必需的。您还需要添加一个实现了 `com.polarion.platform.repository.external.IExternalRepositoryProvider` 接口的插件，以便在您的 Collector 中接收仓库连接配置。其他依赖项（如 `com.polarion.core.util`）是可选的。（注意：请不要创建对 Polarion 平台插件的依赖。）
4. 点击 **Finish**。
5. 创建一个 `plugin.xml` 文件，并为您的 Collector 添加 extension 元素，使其内容如下：

```
<?xml version="1.0" encoding="UTF-8"?>
<?eclipse version="3.0"?>
<plugin>
   <extension point="com.siemens.polarion.rt.collectors">
      <collector
            class="com.example.collectors.CustomCollector"
            repositoryProvider="exampleProvider">
      </collector>
   </extension>
</plugin>
```

实现

实现 `com.siemens.polarion.rt.collectors.api.RtCollector` 接口来定义自定义 Collector。自定义 Collector 的通用算法如下：

1. 使用 context（上下文）初始化采集器。使用 `com.siemens.polarion.rt.collectors.api.RtCollectionContext.getConfiguration()` 获取仓库连接配置，并将该对象转换为特定的 RT 仓库配置接口。打开到仓库的连接。
2. 对配置中指定的每个分支执行以下操作：

3. 确定当前分支在上次采集运行中处理的最后修订版本（revision）。
4. 收集自上次处理的修订版本以来新增、修改和删除的文件信息。如果未定义最后处理的修订版本（例如首次运行时），则收集仓库中所有文件的信息。
5. 对于每个新增或修改的文件，调用 `com.siemens.polarion.rt.collectors.api.RtCollectionContext.shouldBeCollected(RtFilePropertie` 方法来确定该文件是否应由解析器处理。如果该文件应被采集，则通过调用 `com.siemens.polarion.rt.collectors.api.RtCollectionContext.collect(RtFile)` 方法通知采集引擎。对于每个删除的文件和文件夹，调用 `com.siemens.polarion.rt.collectors.api.RtCollectionContext.locationRemoved(String, ILocation)` 方法。
6. 释放已使用的对象。

部署到已安装的 Polarion

请参阅 Polarion SDK 主指南中的"部署到已安装的 Polarion"章节。

从工作区执行

请参阅 Polarion SDK 主指南中的"从工作区执行"章节。

配置

成功将插件部署到 Polarion 后，您可以开始在仓库提供者（repository provider）的 id 与 `plugin.xml` 中指定的 id 相匹配时使用该 Collector。使用上述 `plugin.xml`，可以开发一个与 id 为 `exampleProvider` 的仓库提供者配合使用的 Collector。

在 `repositories.xml` 中添加这样一个仓库：

```
<exampleProvider>
  <id>exampleProviderRepository</id>
  <!-- 仓库设置 -->
</exampleProvider>
```

在 `resource-traceability.xml` 中引用它：

```
<repositoryConfiguration id="exampleRepositoryConfiguration"
repository="exampleProviderRepository">
  <!-- 配置参数 -->
</repositoryConfiguration>
```

自定义 Parser 示例

工作区准备

请参阅 Polarion SDK 主指南中的"工作区准备"章节。

创建项目插件

1. 选择 **File > New > Java Project**。
2. 在出现的对话框中，输入新的插件项目名称，然后点击 **Next** 按钮。
3. 在 **Projects** 选项卡中，将所需的项目添加到构建路径中。插件 `com.siemens.polarion.rt.api` 是开发 Parser 项目所必需的。其他依赖项（如 `com.polarion.core.util`）是可选的。（注意：请不要创建对 Polarion 平台插件的依赖。）
4. 点击 **Finish**。
5. 创建一个 `plugin.xml` 文件，并为 Parser 添加 extension 元素，使其内容如下：

```
<?xml version="1.0" encoding="UTF-8"?>
<?eclipse version="3.0"?>
<plugin>
   <extension
         point="com.siemens.polarion.rt.parsers">
      <parser
            class="com.example.parsers.CustomParser"
            descriptor="com.example.parsers.CustomParserDescriptor"
            name="com.example.parsers.custom">
      </parser>
   </extension>
</plugin>
```

实现

实现 `com.siemens.polarion.rt.parsers.api.RtParser` 接口来定义自定义 Parser。自定义 Parser 的通用算法如下：

1. 使用 `com.siemens.polarion.rt.collectors.api.model.RtFile.getContent()` 获取文件输入流。
2. 读取文件内容并识别文件中的语义元素。根据源文件语言，您可以使用第三方解析器实现从文件内容构建 AST（抽象语法树），或者进行其他转换以创建文件内容模型。
3. 查找包含 Work Item 链接并与内容模型的语义元素相关的注释或其他文本片段。

4. 使用 `com.siemens.polarion.rt.parsers.api.RtParsingContext.parseLinks(String, RtPosition)` 方法从文本中创建链接对象。
5. 构建并返回将被持久化到内部存储中的元素对象。

> **提示**：C 解析器示例插件的源代码可按需提供。请联系 Polarion 支持。

部署到已安装的 Polarion

请参阅 Polarion SDK 主指南中的"部署到已安装的 Polarion"章节。

从工作区执行

请参阅 Polarion SDK 主指南中的"从工作区执行"章节。

配置

成功将插件部署到 Polarion 后，您可以开始为具有特定扩展名的文件使用该 Parser。

例如，在 `resource-traceability.xml` 中添加以下 Parser 配置，以对 `.sql` 文件使用自定义解析器：

```
<fileParser id="customParser" name="com.example.parsers.custom" extensions=".sql">
  <properties>
  </properties>
</fileParser>
```